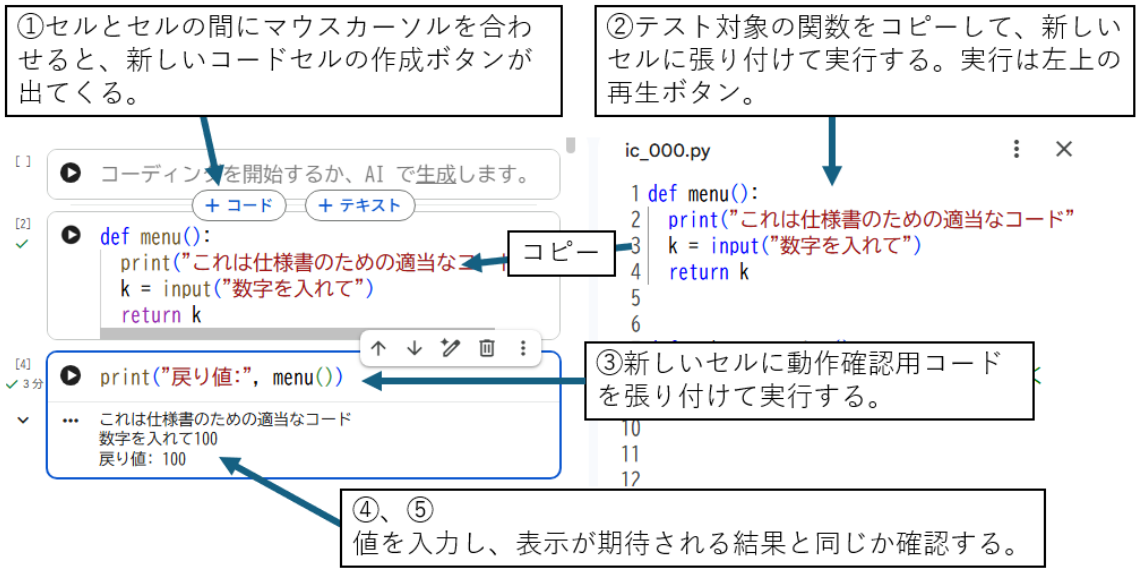
ウチダ電鉄 交通系 IC カード検証システム テスト項目書

テストのやり方（共通）

1. 新しいセルを作る。
2. 本番用のファイルから、テストする関数の部分だけをコピーして、そのセルに貼り付けて実行する。（各関数について、一度実行すれば OK。ただし、関数を修正した場合は再度実行すること）
3. 新しいセルを作成し、「動作確認用コード」を書いて、そのセルを実行する。
4. 入力が必要なケースでは、「入力」の値を上から順に 1 つずつ入力し、Enter を押す。
5. 表示と戻り値が「期待される結果」と同じか確認する。違えば、その関数を見直す。



- 各ケースは独立している。作った関数の章から試せる（第 5 章 main だけは、全ての関数ができてから取り組む）。
- 入力値のチェックは各章の最後のテストケースである。まず正しい入力で動かしてから取り組む。

第 1 章　menu 関数

機能一覧を表示し、選ばれた番号を返す。`99` のときも終了せず、`99` を返すだけである。

動作確認用コード（全ケース共通）

```
print("戻り値:", menu())
```

画面 A

```
【ウチダ電鉄　交通系 IC カード検証システム】

1: 乗車駅選択
2: チャージ機能

使用する機能を入力してください(終了する場合には 99 を入力)
```

ケース	入力	期待される結果
M-01	1	画面 A のあと `戻り値: 1`
M-02	2	画面 A のあと `戻り値: 2`
M-03	99	画面 A のあと `戻り値: 99`（「システムを終了します」は出ない）

入力値のチェック

- **M-04　範囲外の数値**: `-1`、`0`、`3`、`98`、`100` を順に入力する。毎回「正しい数値を入力してください。」と表示され、再入力待ちになる。最後に `1` を入力すると `戻り値: 1`。
- **M-05　数字以外**: `a`、何も入力せず Enter、`1.5` を順に入力する。毎回「数字を入力してください。」と表示され、再入力待ちになる（赤いエラーで止まったら NG）。最後に `2` を入力すると `戻り値: 2`。

第 2 章　select_station 関数

乗車駅を選んでもらい、その駅からの運賃を返す。キャンセル（99）のときは 0 を返す。

動作確認用コード（全ケース共通）

```
print("戻り値:", select_station())
```

画面 B

```
【乗車駅選択】

1: 秋葉原駅から      133 円
2: 山梨駅から       4128 円
3: 長野駅から       7990 円

乗車した駅を入力してください(キャンセルする場合には 99 を入力)
```

ケース	入力	期待される結果（画面 B のあと）
S-01	1	「乗車駅は秋葉原で運賃は 133 円です。」 戻り値: 133
S-02	2	「乗車駅は山梨で運賃は 4128 円です。」 戻り値: 4128
S-03	3	「乗車駅は長野で運賃は 7990 円です。」 戻り値: 7990
S-04	99	「駅の選択をキャンセルしました。」 戻り値: 0

入力値のチェック

- **S-05　範囲外の数値**: -1、0、4、98、100 を順に入力する。毎回「正しい数値を入力してください。」と表示され、再入力待ちになる。最後に 1 を入力すると、S-01 と同じ結果になる。
- **S-06　数字以外**: a、何も入力せず Enter、1.5 を順に入力する。毎回「数字を入力してください。」と表示され、再入力待ちになる（赤いエラーで止まったら NG）。最後に 3 を入力すると、S-03 と同じ結果になる。

第 3 章　pay 関数

チャージ残高から運賃を支払い、支払い後の残高を返す。キーボード入力はないので、入力値のチェックはない（入力欄が出たら NG）。

- 運賃が残高より多い:「残高不足です。」を 1 回表示し、残高が運賃以上になるまで 3000 円ずつ自動チャージする。
- 精算後の残高が 500 円未満: 3000 円を 1 回自動チャージする（ちょうど 500 円ならしない）。

各ケースの動作確認用コードを、新しいセルに書いて実行する。

P-01　残高が十分で、自動チャージなし

```
print("戻り値:", pay(3367, 133))
```

```
チャージ残高は 3367 円です。
精算後のチャージ残高は 3234 円です。
戻り値: 3234
```

P-02　精算後がちょうど 500 円（自動チャージなし）

```
print("戻り値:", pay(633, 133))
```

```
チャージ残高は 633 円です。
精算後のチャージ残高は 500 円です。
戻り値: 500
```

P-03　精算後に 500 円未満になる（仕様書の例）

```
print("戻り値:", pay(500, 133))
```

```
チャージ残高は 500 円です。
精算後のチャージ残高は 367 円です。
残高が 500 円未満のため 3000 円自動チャージします。
チャージ残高は 3367 円です。
戻り値: 3367
```

P-04　精算後が 499 円（自動チャージあり）

```
print("戻り値:", pay(632, 133))
```

```
チャージ残高は 632 円です。
精算後のチャージ残高は 499 円です。
残高が 500 円未満のため 3000 円自動チャージします。
チャージ残高は 3499 円です。
戻り値: 3499
```

P-05　残高と運賃がちょうど同じ

```
print("戻り値:", pay(133, 133))
```

```
チャージ残高は 133 円です。
精算後のチャージ残高は 0 円です。
残高が 500 円未満のため 3000 円自動チャージします。
チャージ残高は 3000 円です。
戻り値: 3000
```

※ 仕様書に記載がないため、残高不足にしない動作を正しいものとする。

P-06　残高が運賃より 1 円足りない

```
print("戻り値:", pay(132, 133))
```

```
チャージ残高は 132 円です。
残高不足です。
3000 円自動チャージします。
精算後のチャージ残高は 2999 円です。
戻り値: 2999
```

P-07　残高不足（自動チャージ 1 回）

```
print("戻り値:", pay(3234, 4128))
```

```
チャージ残高は 3234 円です。
残高不足です。
3000 円自動チャージします。
精算後のチャージ残高は 2106 円です。
戻り値: 2106
```

P-08　残高不足（自動チャージ 2 回・仕様書の例）

```
print("戻り値:", pay(500, 4128))
```

```
チャージ残高は 500 円です。
残高不足です。
3000 円自動チャージします。
3000 円自動チャージします。
精算後のチャージ残高は 2372 円です。
戻り値: 2372
```

P-09　残高不足（自動チャージ 3 回）

```
print("戻り値:", pay(500, 7990))
```

```
チャージ残高は 500 円です。
残高不足です。
3000 円自動チャージします。
3000 円自動チャージします。
3000 円自動チャージします。
精算後のチャージ残高は 1510 円です。
戻り値: 1510
```

P-10　残高不足の自動チャージのあと、精算後にまた 500 円未満

```
print("戻り値:", pay(2106, 7990))
```

```
チャージ残高は 2106 円です。
残高不足です。
3000 円自動チャージします。
3000 円自動チャージします。
精算後のチャージ残高は 116 円です。
残高が 500 円未満のため 3000 円自動チャージします。
チャージ残高は 3116 円です。
戻り値: 3116
```

第 4 章　charge 関数

チャージ残高を受け取り、選ばれた金額をチャージして、チャージ後の残高を返す。キャンセル（99）のときは残高をそのまま返す。

動作確認用コード: `print("戻り値:", charge(500))`（C-02 だけ `charge(3249)`）

画面 C（`charge(500)` のとき）

```
【チャージ機能】

チャージ残高は 500 円です。

1: 1000 円
2: 2000 円
3: 3000 円
4: 4000 円
5: 5000 円
6: 6000 円
7: 7000 円
8: 8000 円
9: 9000 円
10: 10000 円

チャージする金額を選択してください(キャンセルする場合には 99 を入力)
```

ケース	入力	期待される結果（画面 C のあと）
C-01	`1`	「1000 円チャージします。」「チャージ残高は 1500 円です。」 `戻り値: 1500`
C-02	`6`	画面 C の残高は 3249 円。「6000 円チャージします。」「チャージ残高は 9249 円です。」 `戻り値: 9249`
C-03	`10`	「10000 円チャージします。」「チャージ残高は 10500 円です。」 `戻り値: 10500`
C-04	`2` `3` `4` `5` `7` `8` `9` （1 つずつ試す）	`2` なら「2000 円チャージします。」「チャージ残高は 2500 円です。」 `戻り値: 2500`。同様に `3`→3500、`4`→4500、`5`→5500、`7`→7500、`8`→8500、`9`→9500
C-05	`99`	「チャージをキャンセルしました。」 `戻り値: 500`（続く「チャージ残高は 500 円です。」の行は、なくても OK）

入力値のチェック

- **C-06　範囲外の数値**: `-1`、`0`、`11`、`98`、`100` を順に入力する。毎回「正しい数値を入力してください。」と表示され、再入力待ちになる。最後に `1` を入力すると C-01 と同じ結果になる。
- **C-07　数字以外**: `a`、何も入力せず Enter、`1.5` を順に入力する。毎回「数字を入力してください。」と表示され、再入力待ちになる（赤いエラーで止まったら NG）。最後に `10` を入力すると C-03 と同じ結果になる。

第 5 章　main 関数（通しテスト）

全ての関数ができてから行う。うまくいかないときは、先に第 1〜4 章を確認すること。

実行のしかた: 関数をセルに貼らず、全ての関数が入った本番用のファイルをそのまま実行する。

ケース	入力	期待される結果
A-01	`99`	画面 A のあと「システムを終了します」と表示され、終わる

A-02　残高を引き継ぎながら一通り操作する

下の表の入力を、上から順に入力する（`1, 1` は左から 1 つずつ）。操作のあとは、必ず画面 A（メニュー）に戻る。

操作	入力	確認する表示	操作後の残高
秋葉原で乗車	`1, 1`	「チャージ残高は 500 円です。」（最初は 500 円）→「精算後のチャージ残高は 367 円です。」→ 自動チャージ →「チャージ残高は 3367 円です。」	3367
秋葉原で乗車	`1, 1`	「チャージ残高は 3367 円です。」→「精算後のチャージ残高は 3234 円です。」	3234
3000 円チャージ	`2, 3`	画面 C の残高は 3234 円 →「チャージ残高は 6234 円です。」	6234
山梨で乗車	`1, 2`	「精算後のチャージ残高は 2106 円です。」	2106
長野で乗車	`1, 3`	「残高不足です。」→「3000 円自動チャージします。」が 2 回 →「精算後のチャージ残高は 116 円です。」→ 自動チャージ →「チャージ残高は 3116 円です。」	3116
駅の選択をキャンセル	`1, 99`	「駅の選択をキャンセルしました。」だけが出る（精算の表示は出ない）	3116
チャージをキャンセル	`2, 99`	画面 C の残高は 3116 円 →「チャージをキャンセルしました。」	3116

操作	入力	確認する表示	操作後の残高
終了	`99`	「システムを終了します」で終わる	ー

通しで入力する値: `1, 1, 1, 1, 2, 3, 1, 2, 1, 3, 1, 99, 2, 99, 99`

入力値のチェック

- **A-03　メニューでの不正入力**: `3`、`a`、`99` を順に入力する。`3` では「正しい数値を入力してください。」、`a` では「数字を入力してください。」と表示されて再入力待ちになり（アプリは終了しない）、`99` で「システムを終了します」と表示されて終わる。

第 6 章　最終確認チェックリスト

実行ではわからない仕様書のルールを、コードを読んで確認する。第 1〜5 章がすべてできてから、提出前に行うこと。

- ☐ **K-01**　関数名が仕様書通りになっている（`main`、`menu`、`select_station`、`pay`、`charge` から変更していない）
- ☐ **K-02**　引数・戻り値の数と順番が仕様書どおり（`pay(チャージ残高, 運賃)`、`charge(チャージ残高)`）
- ☐ **K-03**　pay の中で charge 関数を呼んでいない
- ☐ **K-04**　数字とアルファベットを全角で書いていない（表示する文字も含む）
- ☐ **K-05**　確認用の `print` を残していない